Linear Data Structures

Lists, Queues, Stacks

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-I





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Course Outline

1. Lists
2. Advanced List Structures
3. Stacks and Queues





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Outline

1 Lists

2 Advanced List Structures

3 Stacks and Queues





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

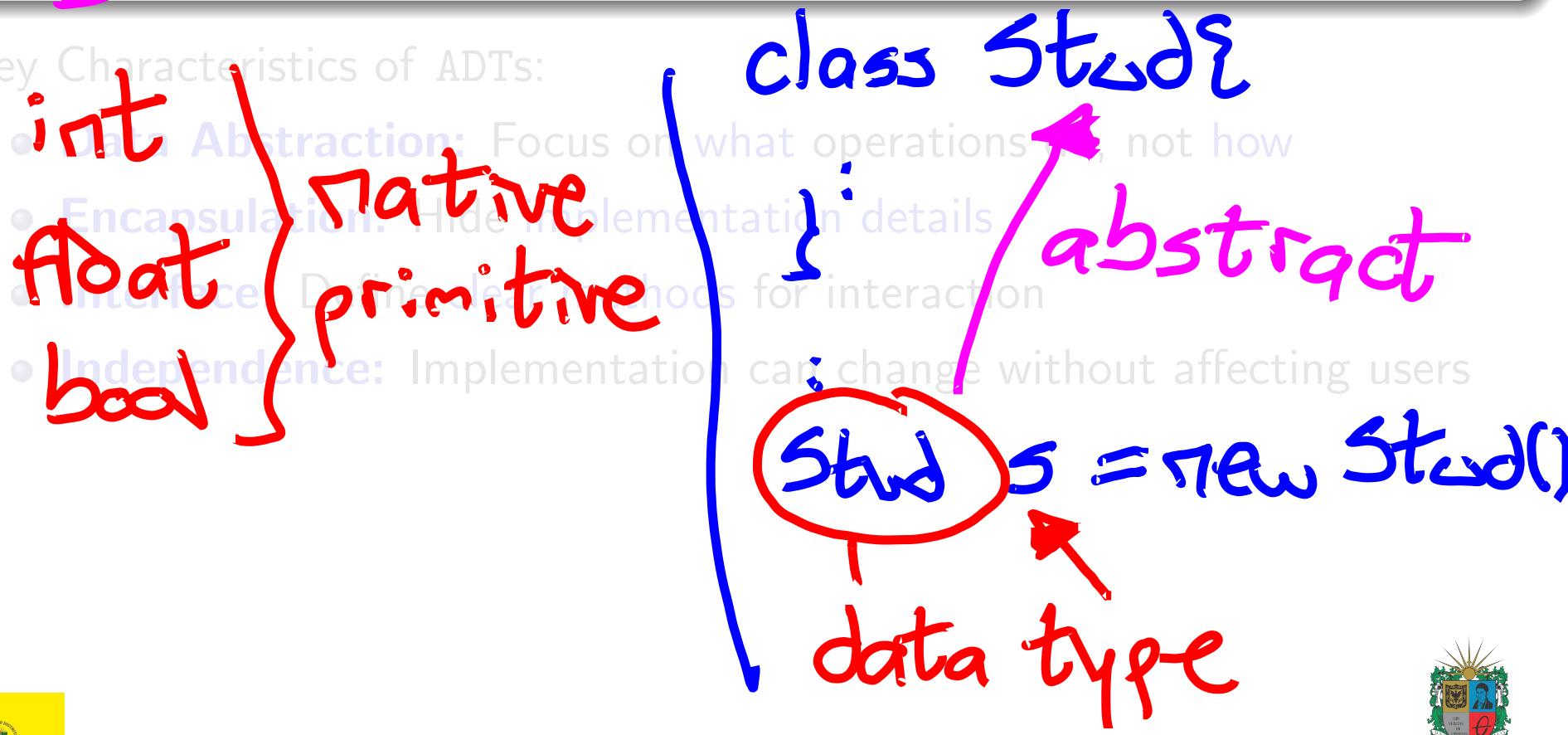
ADT Concept and Definition

Definition

An **Abstract Data Type** (ADT) is a mathematical model that defines data and operations without specifying how they are implemented.

Key Characteristics of ADTs:

- **Data Abstraction:** Focus on what operations do, not how
- **Encapsulation:** Hide implementation details
- **Interface:** Define clear methods for interaction
- **Independence:** Implementation can change without affecting users







UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

ADT Concept and Definition

Definition

An **Abstract Data Type (ADT)** is a mathematical odel that defines data and operations without specifying how they are implemented.

Key Characteristics of ADTs:

- **Data Abstraction:** Focus on what operations do, not how
- **Encapsulation:** Hide implementation details
- **Interface:** Define clear methods for interaction
- **Independence:** Implementation can change without affecting users



ADT Operations

Example

List `ADT` Operations:

- Add element to list
- Remove element from list
- Find element in list
- Get size of list
- Check if list is empty

Update





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Interface vs. Implementation Separation I

Why Separate *Interface* from *Implementation*?

- **Flexibility:** Can change how things work internally
- **Maintainability:** Easier to fix and improve code
- **Reusability:** Same interface, different implementations
- **Testing:** Can test interface independently

Interface (*What*)

- Method signatures
- Expected behavior
- Input/output specifications
- Error conditions

Implementation (*How*)

- Data storage method
- Algorithm details
- Memory management
- Performance optimizations





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Interface vs. Implementation Separation I

Why Separate *Interface* from *Implementation*?

- **Flexibility:** Can change how things work internally
- **Maintainability:** Easier to fix and improve code
- **Reusability:** Same interface, different implementations
- **Testing:** Can test interface independently

Interface (*What*)

- Method signatures
- Expected behavior
- Input/output specifications
- Error conditions

Implementation (*How*)

- Data storage method
- Algorithm details
- Memory management
- Performance optimization





Interface vs. Implementation Separation I

Why Separate *Interface* from *Implementation*?

- **Flexibility:** Can change how things work internally
- **Maintainability:** Easier to fix and improve code
- **Reusability:** Same interface, different implementations
- **Testing:** Can test interface independently

Interface (*What*)

- Method signatures
- Expected behavior
- Input/output specifications
- Error conditions

Implementation (*How*)

- Data storage method
- Algorithm details
- Memory management
- Performance optimizations





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Interface vs. Implementation Separation II

Example

List Interface: `add(item)`, `remove(item)`, `size()`

Implementations: Array-based, Linked-list-based, Dynamic array







Study Case: Student Enrollment System Using Lists I

Problem

A university needs a system to manage student enrollment in courses. Students can be added, removed, and searched in course rosters.

Example

System Requirements:

- Add student to course
- Remove student from course
- Check if student is enrolled
- List all enrolled students
- Get enrollment count
- Transfer student between courses



Study Case: Student Enrollment System Using Lists II

class

EnrollmentSystem

- + addStudent(courseId, student)
- + removeStudent(courseId, student)
- + isEnrolled(courseId, student)
- + getEnrollmentCount(courseId)
- + listStudents(courseId)

A posible solution:

Benefits of Using Lists

- Dynamic size as enrollment changes
- Easy to add/remove students
- Can maintain enrollment order
- Efficient searching capabilities





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Study Case: Student Enrollment System Using Lists II

A posible solution:

EnrollmentSystem
+ addStudent(courseId, student)
+ removeStudent(courseId, student)
+ isEnrolled(courseId, student)
+ getEnrollmentCount(courseId)
+ listStudents(courseId)

Benefits of Using Lists

- Dynamic size as enrollment changes
- Easy to add/remove students
- Can maintain enrollment order
- Efficient searching capabilities





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Static vs. Dynamic Lists I

Static Lists

Characteristics:

- Fixed size at creation
- Uses arrays as base structure
- Memory allocated at compile time
- Fast access by index

Dynamic Lists

Characteristics:

- Size changes during execution
- Uses pointers or expandable arrays
- Memory allocated at runtime
- Flexible size management

When to Use Each

- Static: Known size, frequent access, simple operations
- Dynamic: Unknown size, frequent insertions/deletions





Static vs. Dynamic Lists I

Static Lists

Characteristics:

- Fixed size at creation
- Uses arrays as base structure
- Memory allocated at compile time
- Fast access by index

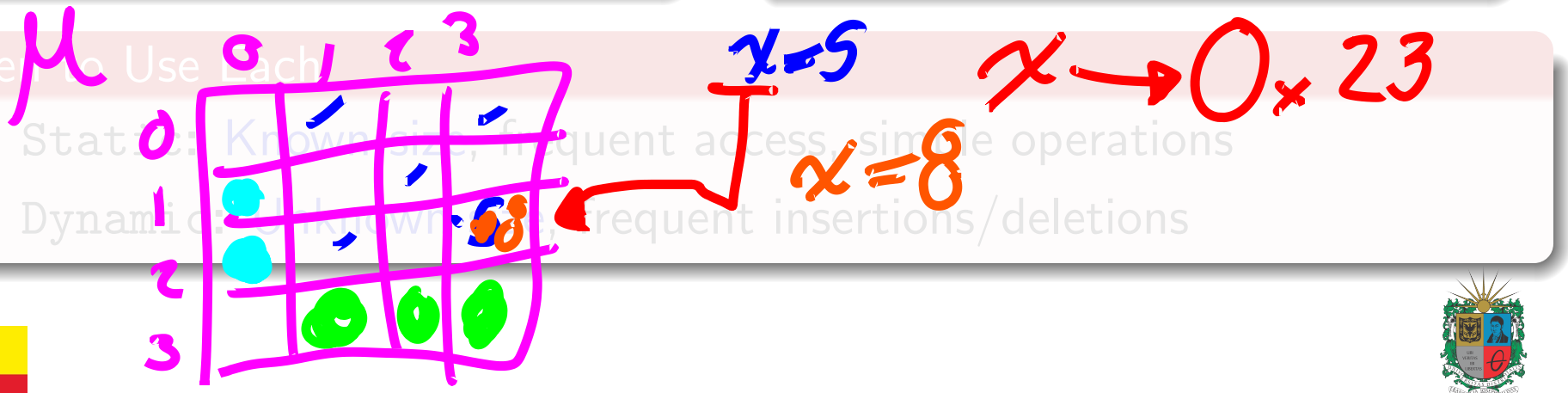
Dynamic Lists

Characteristics:

- Size changes during execution
- Uses pointers or expandable arrays
- Memory allocated at runtime
- Flexible size management

When to Use Each?

- Static: Known size, frequent access, simple operations
- Dynamic: Unknown size, frequent insertions/deletions





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Static vs. Dynamic Lists I

Static Lists

Characteristics:

- Fixed size at creation
- Uses arrays as base structure
- Memory allocated at compile time
- Fast access by index

Dynamic Lists

Characteristics:

- Size changes during execution
- Uses pointers or expandable arrays
- Memory allocated at runtime
- Flexible size management

When to Use Each

- `Static`: Known size, frequent access, simple operations
- `Dynamic`: Unknown size, frequent insertions/deletions





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Static vs. Dynamic Lists II

Static Lists

Advantages:

- Simple implementation
- Fast random access
- No memory overhead

Disadvantages:

- Fixed maximum size
- Memory waste if not full
- Insertion/deletion expensive

Dynamic Lists

Advantages:

- No size limitations
- Efficient memory usage
- Easy insertion/deletion

Disadvantages:

- More complex implementation
- Memory overhead for pointers
- Slower access in some cases





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Static vs. Dynamic Lists II

Static Lists

Advantages:

- Simple implementation
- Fast random access
- No memory overhead

Disadvantages:

- Fixed maximum size
- Memory waste if not full
- Insertion/deletion expensive

Dynamic Lists

Advantages:

- No size limitations
- Efficient memory usage
- Easy insertion/deletion

Disadvantages:

- More complex implementation
- Memory overhead for pointers
- Slower access in some cases





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

List Operations I: Insert, Search

Core List Operations

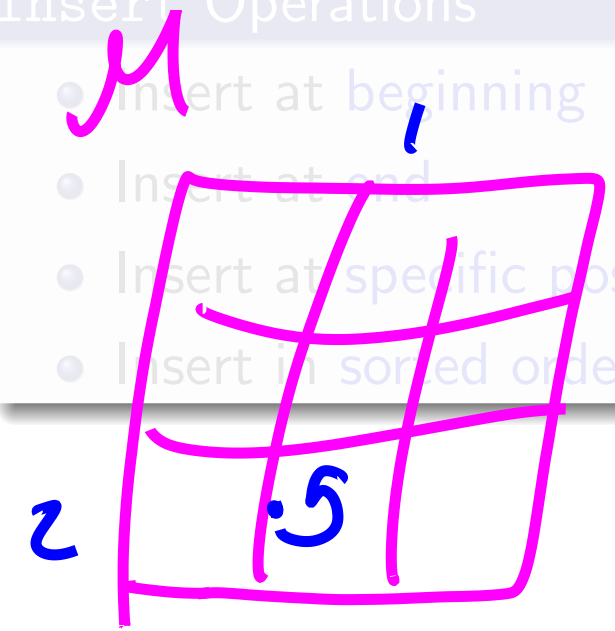
All list implementations must support fundamental operations for data manipulation.

Insert Operations

- Insert at beginning
- Insert at end
- Insert at specific position
- Insert in sorted order

Search Operations

- Find element by value
- Find element at position
- Check if element exists
- Count occurrences



List Operations I: `Insert`, `Search`

Core List Operations

All list implementations must support fundamental operations for data manipulation.

`Insert` Operations

- Insert at beginning
- Insert at end
- Insert at specific position
- Insert in sorted order

`Search` Operations

- Find element by value
- Find element at position
- Check if element exists
- Count occurrences





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

List Operations I: `Insert`, `Search`

Core List Operations

All list implementations must support fundamental operations for data manipulation.

`Insert` Operations

- Insert at beginning
- Insert at end
- Insert at specific position
- Insert in sorted order

`Search` Operations

- Find element by value
- Find element at position
- Check if element exists
- Count occurrences

bool





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

List Operations II: `Delete`, `Update`

Core List Operations

All list implementations must support fundamental operations for data manipulation.

`Delete` Operations

- Delete by value
- Delete at position
- Delete first occurrence
- Delete all occurrences

`Update` Operations

- Update element at position
- Update first occurrence
- Update all occurrences
- Swap elements





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

List Operations II: `Delete`, `Update`

Core List Operations

All list implementations must support fundamental operations for data manipulation.

`Delete` Operations

- Delete by value
- Delete at position
- Delete first occurrence
- Delete all occurrences

`Update` Operations

- Update element at position
- Update first occurrence
- Update all occurrences
- Swap elements

C - create (insert)
R - read/retrieve (search)
U - update
D - delete





Study Case: `Shopping Cart` Implementation with *Array-based List* I

Problem

Implement a shopping cart system where customers can add items, remove items, update quantities, and view their cart contents.

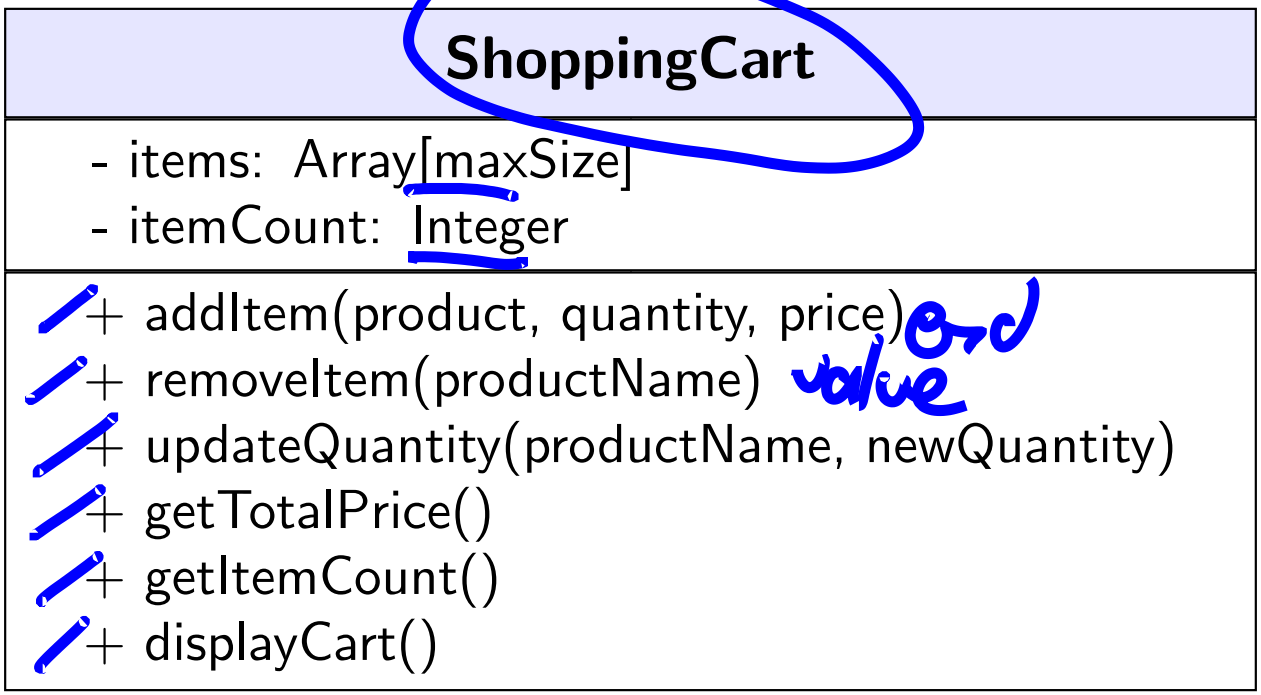


maximum amount

Study Case: `Shopping Cart` Implementation with *Array-based List* I

Problem

Implement a shopping cart system where customers can add items, remove items, update quantities, and view their cart contents.



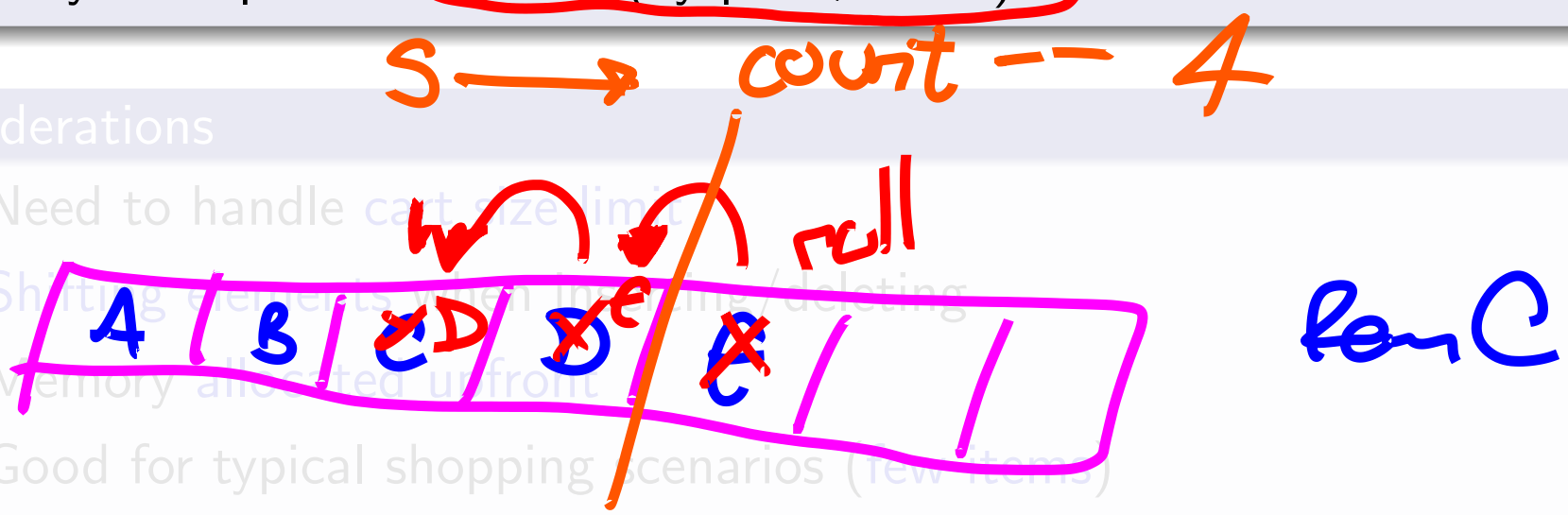
Study Case: Shopping Cart Analysis with *Array-based List* II

Array Implementation Benefits

- Fast access to any item by index
- Simple memory layout
- Good for small, bounded carts
- Easy to implement sorting (by price, name)

Considerations

- Need to handle cart size limits
- Shifting elements when inserting/deleting
- Memory allocated upfront
- Good for typical shopping scenarios (few items)



Study Case: `Shopping Cart` Analysis with *Array-based List* II

Array Implementation Benefits

- Fast access to any item by index
- Simple memory layout
- Good for small, bounded carts
- Easy to implement sorting (by price, name)

Considerations

- Need to handle cart size limit
- Shifting elements when inserting/deleting
- Memory allocated upfront
- Good for typical shopping scenarios (few items)





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

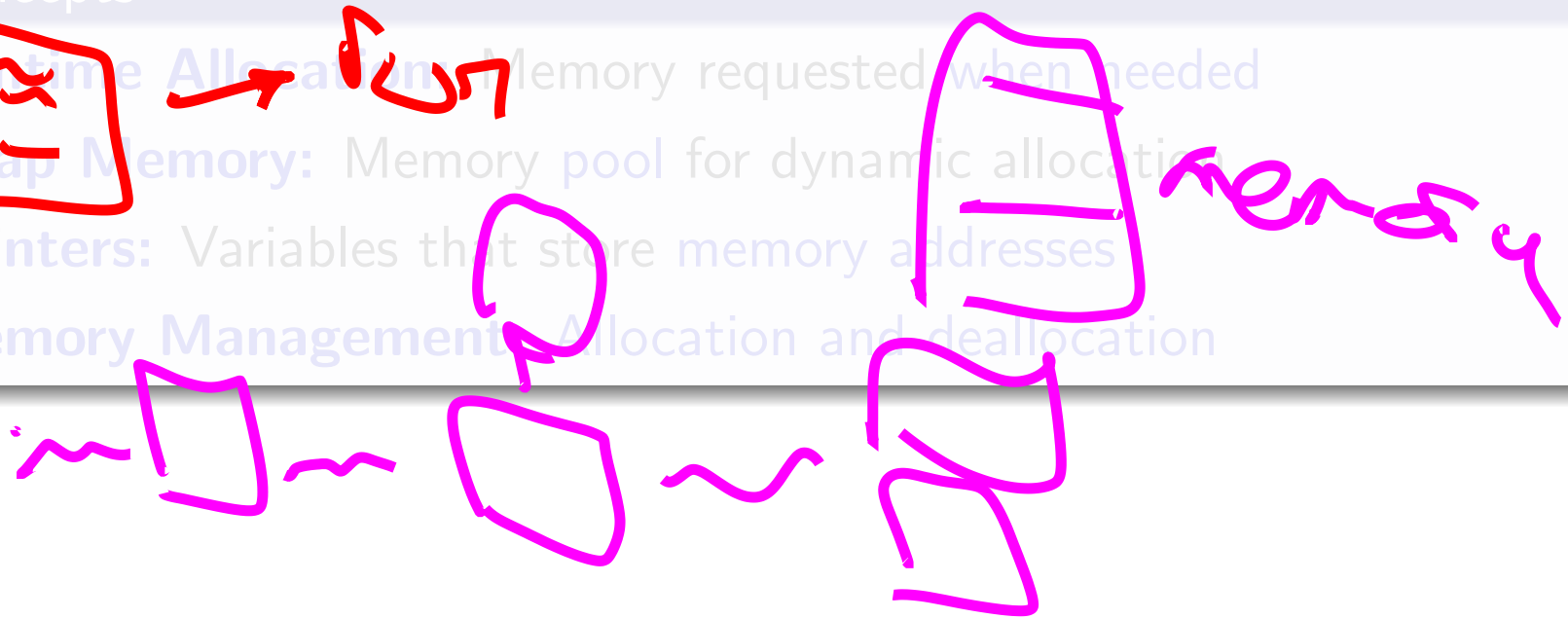
Dynamic Memory Allocation I

Definition

Dynamic memory allocation allows programs to **request memory during execution**, enabling data structures to **grow and shrink** as needed.

Key Concepts

- **Runtime Allocation:** Memory requested when needed
- **Heap Memory:** Memory pool for dynamic allocation
- **Pointers:** Variables that store memory addresses
- **Memory Management:** Allocation and deallocation







Dynamic Memory Allocation I

Definition

Dynamic memory allocation allows programs to request memory during execution, enabling data structures to grow and shrink as needed.

Key Concepts

- **Runtime Allocation:** Memory requested when needed
- **Heap Memory:** Memory pool for dynamic allocation
- **Pointers:** Variables that store memory addresses
- **Memory Management:** Allocation and deallocation





Dynamic Memory Allocation II

Advantages

- Flexible memory usage
- No size limits at compile time
- Memory used only when needed
- Can handle varying data sizes

Challenges

- Memory leaks possible
- Fragmentation issues
- Slower than static allocation
- More complex programming

Best Practices

- Always free allocated memory
- Check for allocation failures
- Avoid memory leaks
- Use appropriate allocation strategies

Dynamic Memory Allocation II

Advantages

- Flexible memory usage
- No size limits at compile time
- Memory used only when needed
- Can handle varying data sizes

Challenges

- Memory leaks possible
- Fragmentation issues
- Slower than static allocation
- More complex programming

Best Practices

- Always free allocated memory
- Check for allocation failures
- Avoid memory leaks
- Use appropriate allocation strategies

Dynamic Memory Allocation II

Advantages

- Flexible memory usage
- No size limits at compile time
- Memory used only when needed
- Can handle varying data sizes

Challenges

- Memory leaks possible
- Fragmentation issues
- Slower than static allocation
- More complex programming

Best Practices

- Always free allocated memory
- Check for allocation failures
- Avoid memory leaks
- Use appropriate allocation strategies

Pointer-based List Structures I

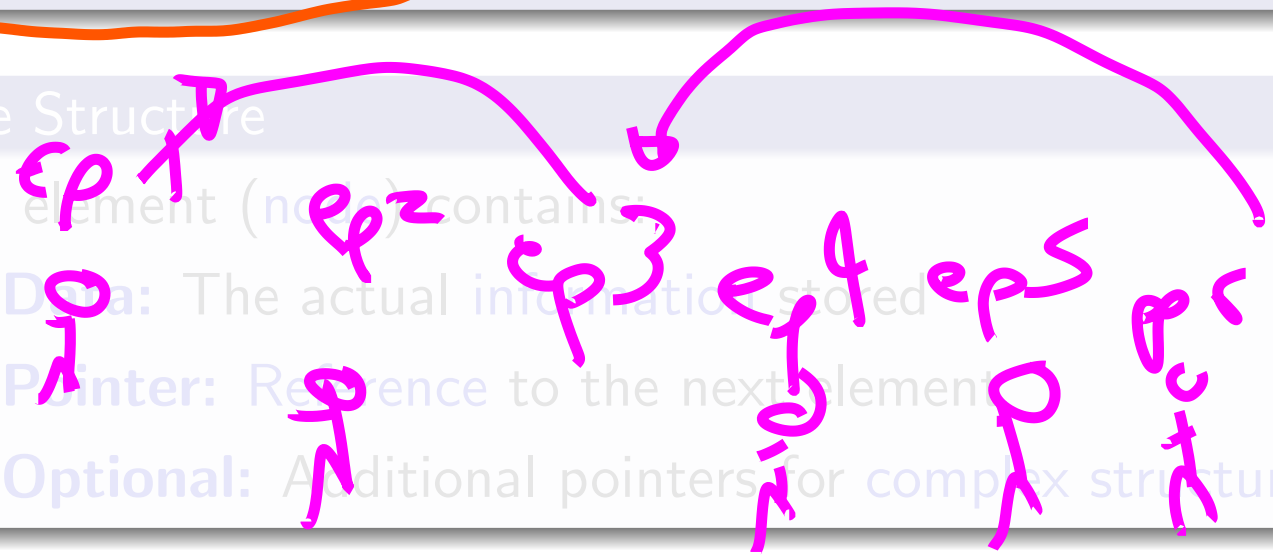
Definition

Pointer-based lists use dynamic memory allocation and pointers to create flexible data structures where elements can be stored anywhere in memory.

Node Structure

Each element (node) contains:

- **Data:** The actual information stored
- **Pointer:** Reference to the next element
- **Optional:** Additional pointers for complex structures



Pointer-based List Structures I

Definition

Pointer-based lists use dynamic memory allocation and pointers to create flexible data structures where elements can be stored anywhere in memory.

Node Structure

Each element (node) contains:

- **Data:** The actual information stored
- **Pointer:** Reference to the next element
- **Optional:** Additional pointers for complex structures





Pointer-based List Structures II

Structure Node
data: DataType
next: Pointer to Node
end structure

Class LinkedList
head: Pointer to Node
size: Integer





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Pointer-based List Structures III

Advantages

- Dynamic size
- Efficient insertion/deletion
- Memory allocated as needed
- No memory waste

Disadvantages

- No random access
- Extra memory for pointers
- Sequential access only
- More complex implementation





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Pointer-based List Structures III

Advantages

- Dynamic size
- Efficient insertion/deletion
- Memory allocated as needed
- No memory waste

Disadvantages

- No random access
- Extra memory for pointers
- Sequential access only
- More complex implementation





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Study Case: `Music Playlist Management` with *Pointer-based List* I

Problem

Create a music playlist system where songs can be easily added, removed, and reordered. The playlist should handle an unknown number of songs efficiently.

1: **Structure** Song
2: title: String
3: artist: String
4: duration: Integer
5: next: Pointer to Song
6: **end structure**

1: **Class** Playlist
2: head: Pointer to Song
3: current: Pointer to Song
4: totalSongs: Integer





Study Case: `Music Playlist Management` with *Pointer-based List* I

Problem

Create a music playlist system where songs can be easily added, removed, and reordered. The playlist should handle an unknown number of songs efficiently.

1: **Structure** Song
2: title: String
3: artist: String
4: duration: Integer
5: next: Pointer to Song
6: **end structure**

1: **Class** Playlist
2: head: Pointer to Song
3: current: Pointer to Song
4: totalSongs: Integer





Study Case: `Music Playlist Management` with *Pointer-based List* II

Playlist Operations

- **Add Song:** Insert at end or specific position
- **Remove Song:** Delete by title or position
- **Play Next:** Move to next song in sequence
- **Shuffle:** Rearrange song order randomly
- **Search:** Find song by title or artist

Why Linked List Works Well

- Unknown number of songs
- Frequent additions/removals
- Sequential playback matches structure
- Easy to implement shuffle and repeat

Study Case: `Music Playlist Management` with *Pointer-based List* II

Playlist Operations

- **Add Song:** Insert at end or specific position
- **Remove Song:** Delete by title or position
- **Play Next:** Move to next song in sequence
- **Shuffle:** Rearrange song order randomly
- **Search:** Find song by title or artist

Why Linked List Works Well

- Unknown number of songs
- Frequent additions/removals
- Sequential playback matches structure
- Easy to implement shuffle and repeat

Outline

1 Lists

2 Advanced List Structures

3 Stacks and Queues





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Double-linked Lists: Structure and Advantages

Definition

A double-linked list is a linked data structure where each node contains pointers to both the next and previous nodes, enabling bidirectional traversal.

```
1: Structure DoubleNode
2: data: DataType
3: next: Pointer to DoubleNode
4: previous: Pointer to DoubleNode
5: end structure
6:
7: Class DoubleLinkedList
8: head: Pointer to DoubleNode
9: tail: Pointer to DoubleNode
10: size: Integer
```





Double-linked Lists: Structure and Advantages

Definition

A double-linked list is a linked data structure where each node contains pointers to both the next and previous nodes, enabling bidirectional traversal.

```
1: Structure DoubleNode
2: data: DataType
3: next: Pointer to DoubleNode
4: previous: Pointer to DoubleNode
5: end structure
6:
7: Class DoubleLinkedList
8: head: Pointer to DoubleNode
9: tail: Pointer to DoubleNode
10: size: Integer
```





Double-linked Lists: Structure and Advantages

Advantages

- Bidirectional navigation
- Efficient deletion
- Easy insertion at end
- Better for undo operations
- Supports reverse iteration

Disadvantages

- Extra memory for pointers
- More complex implementation
- Increased maintenance overhead
- Slower single insertions

When to Use

- Need backward navigation
- Frequent deletions
- Undo/redo functionality
- Browser-like navigation



Double-linked Lists: Structure and Advantages

Advantages

- Bidirectional navigation
- Efficient deletion
- Easy insertion at end
- Better for undo operations
- Supports reverse iteration

Disadvantages

- Extra memory for pointers
- More complex implementation
- Increased maintenance overhead
- Slower single insertions

When to Use

- Need backward navigation
- Frequent deletions
- Undo/redo functionality
- Browser-like navigation



Bidirectional Traversal Benefits I

Forward and Backward Movement

`Double-linked` lists allow efficient movement in both directions, opening up new possibilities for data access patterns.

Example

Traversal Operations:

- **Forward:** Start from head, follow `next` pointers
- **Backward:** Start from tail, follow `previous` pointers
- **Search from both ends:** Find middle element faster
- **Range operations:** Process elements in any direction





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Bidirectional Traversal Benefits I

Forward and Backward Movement

`Double-linked` lists allow efficient movement in both directions, opening up new possibilities for data access patterns.

Example

Traversal Operations:

- **Forward:** Start from head, follow `next` pointers
- **Backward:** Start from tail, follow `previous` pointers
- **Search from both ends:** Find middle element faster
- **Range operations:** Process elements in any direction





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Bidirectional Traversal Benefits II

Practical Benefits

- **Faster Search:** Search from nearest end
- **Easy Deletion:** No need to find previous node
- **Flexible Iteration:** Support both directions
- **Better User Experience:** Natural navigation patterns

```
1: method findFromNearestEnd(value)
2: if position closer to head then
3:     search forward from head
4: else
5:     search backward from tail
6: end if
```





Bidirectional Traversal Benefits II

Practical Benefits

- **Faster Search:** Search from nearest end
- **Easy Deletion:** No need to find previous node
- **Flexible Iteration:** Support both directions
- **Better User Experience:** Natural navigation patterns

```
1: method findFromNearestEnd(value)
2: if position closer to head then
3:     search forward from head
4: else
5:     search backward from tail
6: end if
```





Study Case: Browser Navigation

Problem

Implement browser navigation history where users can go back and forward through visited pages efficiently.

```
1: Structure WebPage
2: url: String
3: title: String
4: timestamp: DateTime
5: next: Pointer to WebPage
6: previous: Pointer to
   WebPage
7: end structure
```

```
1: Class BrowserHistory
2: currentPage: Pointer to
   WebPage
3: historySize: Integer
```





Study Case: Browser Navigation

Problem

Implement browser navigation history where users can go back and forward through visited pages efficiently.

```
1: Structure WebPage
2: url: String
3: title: String
4: timestamp: DateTime
5: next: Pointer to WebPage
6: previous: Pointer to WebPage
7: end structure
```

```
1: Class BrowserHistory
2: currentPage: Pointer to WebPage
3: historySize: Integer
```





Study Case: Browser Navigation

Browser Operations

- **Visit Page:** Add new page, become current
- **Go Back:** Move to previous page
- **Go Forward:** Move to next page
- **Clear History:** Remove all pages
- **Show History:** Display visited pages

Why Double-Linked List is Perfect

- Natural back/forward navigation
- Easy to implement browser buttons
- Efficient page switching
- Maintains chronological order
- Supports history size limits



Study Case: Browser Navigation

Browser Operations

- **Visit Page:** Add new page, become current
- **Go Back:** Move to previous page
- **Go Forward:** Move to next page
- **Clear History:** Remove all pages
- **Show History:** Display visited pages

Why Double-Linked List is Perfect

- Natural back/forward navigation
- Easy to implement browser buttons
- Efficient page switching
- Maintains chronological order
- Supports history size limits



Multi-lists: Definition and Applications I

Definition

Multi-lists are data structures where elements can belong to multiple lists simultaneously through different pointer sets.

Structure Concept

Each node contains:

- **Data:** The stored information
- **Multiple Pointers:** Links to different lists
- **List Identifiers:** Which lists the node belongs to





Multi-lists: Definition and Applications I

Definition

Multi-lists are data structures where elements can belong to multiple lists simultaneously through different pointer sets.

Structure Concept

Each node contains:

- **Data:** The stored information
- **Multiple Pointers:** Links to different lists
- **List Identifiers:** Which lists the node belongs to





Multi-lists: Definition and Applications II

1: **Structure** MultiNode
2: data: DataType
3: nextByCategory: Pointer to MultiNode
4: nextByPriority: Pointer to MultiNode
5: nextByDate: Pointer to MultiNode
6: **end structure**

Applications

- **Database Indexing:** Multiple sort orders
- **Task Management:** Categorize by priority, date, type
- **Social Networks:** Friend lists, interest groups
- **Library Systems:** Books sorted by author, subject, date
- **Email Systems:** Inbox, sent, important, spam





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Multi-lists: Definition and Applications II

1: **Structure** MultiNode
2: data: DataType
3: nextByCategory: Pointer to MultiNode
4: nextByPriority: Pointer to MultiNode
5: nextByDate: Pointer to MultiNode
6: **end structure**

Applications

- **Database Indexing:** Multiple sort orders
- **Task Management:** Categorize by priority, date, type
- **Social Networks:** Friend lists, interest groups
- **Library Systems:** Books sorted by author, subject, date
- **Email Systems:** Inbox, sent, important, spam





Circular Lists: Implementation Considerations I

Definition

A `Circular list` is a linked list where the last node points back to the first node, creating a circular structure.

Types of Circular Lists

- `Singly Circular:` Last node points to first
- `Doubly Circular:` Both `next` and `previous` links wrap around
- **Circular with Header:** Special header node for easy manipulation





Circular Lists: Implementation Considerations I

Definition

A `Circular list` is a linked list where the last node points back to the first node, creating a circular structure.

Types of Circular Lists

- `Singly Circular:` Last node points to first
- `Doubly Circular:` Both `next` and `previous` links wrap around
- **Circular with Header:** Special header node for easy manipulation





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Circular Lists: Implementation Considerations II

Implementation Considerations

- **Termination Condition:** How to detect end of traversal
- **Empty List Handling:** Special case management
- **Insertion/Deletion:** Maintain circular property
- **Memory Management:** Avoid infinite loops in cleanup

```
1: method traverse()
2: current = head
3: do
4: process(current.data)
5: current = current.next
6: while current != head
```

Common Use Cases

- Round-robin scheduling
- Circular buffers

Circular Lists: Implementation Considerations II

Implementation Considerations

- **Termination Condition:** How to detect end of traversal
- **Empty List Handling:** Special case management
- **Insertion/Deletion:** Maintain circular property
- **Memory Management:** Avoid infinite loops in cleanup

```
1: method traverse()
2: current = head
3: do
4: process(current.data)
5: current = current.next
6: while current != head
```

Common Use Cases

- Round-robin scheduling
- Circular buffers

Circular Lists: Implementation Considerations II

Implementation Considerations

- **Termination Condition:** How to detect end of traversal
- **Empty List Handling:** Special case management
- **Insertion/Deletion:** Maintain circular property
- **Memory Management:** Avoid infinite loops in cleanup

```
1: method traverse()
2: current = head
3: do
4: process(current.data)
5: current = current.next
6: while current != head
```

Common Use Cases

- Round-robin scheduling
- Circular buffers

Study Case: Round-robin Scheduling with Circular Lists I

Problem

Implement a round-robin scheduler for processes where each process gets equal CPU time in a circular fashion.

```
1:  Structure Process
2:  processId: Integer
3:  name: String
4:  remainingTime: Integer
5:  priority: Integer
6:  next: Pointer to Process
7:  end structure
8:
9:  Class RoundRobinScheduler
10: currentProcess: Pointer to Process
11: timeQuantum: Integer
12: totalProcesses: Integer
```





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Study Case: Round-robin Scheduling with Circular Lists I

Problem

Implement a round-robin scheduler for processes where each process gets equal CPU time in a circular fashion.

```
1:  Structure Process
2:  processId: Integer
3:  name: String
4:  remainingTime: Integer
5:  priority: Integer
6:  next: Pointer to Process
7:  end structure
8:
9:  Class RoundRobinScheduler
10: currentProcess: Pointer to Process
11: timeQuantum: Integer
12: totalProcesses: Integer
```





Study Case: Round-robin Scheduling with Circular Lists I

Scheduler Operations

- **Add Process:** Insert new process into circular queue
- **Execute Quantum:** Run current process for time slice
- **Switch Process:** Move to next process in circle
- **Remove Completed:** Remove finished processes
- **Get Status:** Show current scheduling state

Benefits of Circular List

- Automatic rotation through processes
- Fair time allocation
- Simple implementation
- Easy to add/remove processes
- Natural round-robin behavior

Study Case: `Round-robin Scheduling` with Circular Lists I

Scheduler Operations

- **Add Process:** Insert new process into circular queue
- **Execute Quantum:** Run current process for time slice
- **Switch Process:** Move to next process in circle
- **Remove Completed:** Remove finished processes
- **Get Status:** Show current scheduling state

Benefits of `Circular` List

- Automatic rotation through processes
- Fair time allocation
- Simple implementation
- Easy to add/remove processes
- Natural round-robin behavior

Object-oriented List Design Principles

Design Principles for List Classes

Applying good object-oriented design makes lists more maintainable, reusable, and extensible.

Core Principles

- **Encapsulation:** Hide implementation details
- **Abstraction:** Provide clear interfaces
- **Inheritance:** Share common functionality
- **Polymorphism:** Same interface, different behavior

1: **interface** List
2: **method** add(element)

Design Guidelines

- Define clear interfaces
- Separate concerns properly
- Use composition when appropriate
- Implement error handling
- Consider thread safety





Object-oriented List Design Principles

Design Principles for List Classes

Applying good object-oriented design makes lists more maintainable, reusable, and extensible.

Core Principles

- **Encapsulation:** Hide implementation details
- **Abstraction:** Provide clear interfaces
- **Inheritance:** Share common functionality
- **Polymorphism:** Same interface, different behavior

Design Guidelines

- Define clear interfaces
- Separate concerns properly
- Use composition when appropriate
- Implement error handling
- Consider thread safety



1. Interface List
2. method add(element)



Object-oriented List Design Principles

Design Principles for List Classes

Applying good object-oriented design makes lists more maintainable, reusable, and extensible.

Core Principles

- **Encapsulation:** Hide implementation details
- **Abstraction:** Provide clear interfaces
- **Inheritance:** Share common functionality
- **Polymorphism:** Same interface, different behavior

Design Guidelines

- Define clear interfaces
- Separate concerns properly
- Use composition when appropriate
- Implement error handling
- Consider thread safety



1: **Interface** List
2: **method** add(element)



Object-oriented List Design Principles

Design Principles for List Classes

Applying good object-oriented design makes lists more maintainable, reusable, and extensible.

Core Principles

- **Encapsulation:** Hide implementation details
- **Abstraction:** Provide clear interfaces
- **Inheritance:** Share common functionality
- **Polymorphism:** Same interface, different behavior

Design Guidelines

- Define clear interfaces
- Separate concerns properly
- Use composition when appropriate
- Implement error handling
- Consider thread safety



1: **Interface** List
2: **method** add(element)



Class Hierarchies for Different List Types

Building a List Class Hierarchy

Organize different list implementations into a coherent class structure that promotes code reuse and maintainability.

```
1: Abstract Class AbstractList
2: size: Integer
3: abstract method add(element)
4: abstract method remove(element)
5: method isEmpty() return size == 0
6: method getSize() return size
7:
8: Class ArrayList extends AbstractList
9: Class LinkedList extends AbstractList
10: Class DoubleLinkedList extends AbstractList
11: Class CircularList extends AbstractList
```



Hierarchy Benefits

Class Hierarchies for Different List Types

Building a List Class Hierarchy

Organize different list implementations into a coherent class structure that promotes code reuse and maintainability.

1: **Abstract Class** AbstractList
2: size: Integer
3: **abstract method** add(element)
4: **abstract method** remove(element)
5: **method** isEmpty() return size == 0
6: **method** getSize() return size
7:
8: **Class** ArrayList extends AbstractList
9: **Class** LinkedList extends AbstractList
10: **Class** DoubleLinkedList extends AbstractList
11: **Class** CircularList extends AbstractList



Hierarchy Benefits

Class Hierarchies for Different List Types

Building a List Class Hierarchy

Organize different list implementations into a coherent class structure that promotes code reuse and maintainability.

1: **Abstract Class** AbstractList
2: size: Integer
3: **abstract method** add(element)
4: **abstract method** remove(element)
5: **method** isEmpty() return size == 0
6: **method** getSize() return size
7:
8: **Class** ArrayList extends AbstractList
9: **Class** LinkedList extends AbstractList
10: **Class** DoubleLinkedList extends AbstractList
11: **Class** CircularList extends AbstractList



Hierarchy Benefits

Class Hierarchies for Different List Types

Building a List Class Hierarchy

Organize different list implementations into a coherent class structure that promotes code reuse and maintainability.

1: **Abstract Class** AbstractList
2: size: Integer
3: **abstract method** add(element)
4: **abstract method** remove(element)
5: **method** isEmpty() return size == 0
6: **method** getSize() return size
7:
8: **Class** ArrayList extends AbstractList
9: **Class** LinkedList extends AbstractList
10: **Class** DoubleLinkedList extends AbstractList
11: **Class** CircularList extends AbstractList



Hierarchy Benefits

Study Case: Professional List Library Design

Problem

Design a professional-grade list library that can be used in various applications with different performance requirements and use cases.

```
1: Package DataStructures.Lists
2:
3: Interface List¡T¿
4: Interface IndexedList¡T¿ extends List¡T¿
5: Interface SortedList¡T¿ extends List¡T¿
6:
7: Class ArrayList¡T¿ implements IndexedList¡T¿
8: Class LinkedList¡T¿ implements List¡T¿
9: Class TreeList¡T¿ implements SortedList¡T¿
```

Library Features

- **Generic Types:** Work with any data type
- **Multiple Interfaces:** Different access patterns

Study Case: Professional List Library Design

Problem

Design a professional-grade list library that can be used in various applications with different performance requirements and use cases.

```
1: Package DataStructures.Lists
2:
3: Interface List¡T¿
4: Interface IndexedList¡T¿ extends List¡T¿
5: Interface SortedList¡T¿ extends List¡T¿
6:
7: Class ArrayList¡T¿ implements IndexedList¡T¿
8: Class LinkedList¡T¿ implements List¡T¿
9: Class TreeList¡T¿ implements SortedList¡T¿
```

Library Features

- **Generic Types:** Work with any data type
- **Multiple Interfaces:** Different access patterns

Study Case: Professional List Library Design

Problem

Design a professional-grade list library that can be used in various applications with different performance requirements and use cases.

```
1: Package DataStructures.Lists
2:
3: Interface List¡T¿
4: Interface IndexedList¡T¿ extends List¡T¿
5: Interface SortedList¡T¿ extends List¡T¿
6:
7: Class ArrayList¡T¿ implements IndexedList¡T¿
8: Class LinkedList¡T¿ implements List¡T¿
9: Class TreeList¡T¿ implements SortedList¡T¿
```

Library Features

- **Generic Types:** Work with any data type
- **Multiple Interfaces:** Different access patterns

Study Case: Professional List Library Design

Problem

Design a professional-grade list library that can be used in various applications with different performance requirements and use cases.

```
1: Package DataStructures.Lists
2:
3: Interface List¡T¿
4: Interface IndexedList¡T¿ extends List¡T¿
5: Interface SortedList¡T¿ extends List¡T¿
6:
7: Class ArrayList¡T¿ implements IndexedList¡T¿
8: Class LinkedList¡T¿ implements List¡T¿
9: Class TreeList¡T¿ implements SortedList¡T¿
```

Library Features

- **Generic Types:** Work with any data type
- **Multiple Interfaces:** Different access patterns

Outline

1 Lists

2 Advanced List Structures

3 Stacks and Queues





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Stack: LIFO Principle and Operations

Definition

A stack is a linear data structure that follows the Last-In-First-Out (LIFO) principle, where elements are added and removed from the same end.

LIFO Principle

- **Last In:** The most recently added element
- **First Out:** The first element to be removed
- **Access Point:** Only the top element is accessible
- **Analogy:** Stack of plates or books

Basic Stack Operations

- **Push:** Add element to top of stack
- **Pop:** Remove and return top element
- **Peek/Top:** View top element without removing
- **IsEmpty:** Check if stack has no elements

Stack: LIFO Principle and Operations

Definition

A stack is a linear data structure that follows the Last-In-First-Out (LIFO) principle, where elements are added and removed from the same end.

LIFO Principle

- **Last In:** The most recently added element
- **First Out:** The first element to be removed
- **Access Point:** Only the top element is accessible
- **Analogy:** Stack of plates or books

Basic Stack Operations

- **Push:** Add element to top of stack
- **Pop:** Remove and return top element
- **Peek/Top:** View top element without removing
- **IsEmpty:** Check if stack has no elements

Stack: LIFO Principle and Operations

Definition

A stack is a linear data structure that follows the Last-In-First-Out (LIFO) principle, where elements are added and removed from the same end.

LIFO Principle

- **Last In:** The most recently added element
- **First Out:** The first element to be removed
- **Access Point:** Only the top element is accessible
- **Analogy:** Stack of plates or books

Basic Stack Operations

- `Push`: Add element to top of stack
- `Pop`: Remove and return top element
- `Peek`/**Top:** View top element without removing
- **IsEmpty:** Check if stack has no elements

Stack: LIFO Principle and Operations

Definition

A stack is a linear data structure that follows the Last-In-First-Out (LIFO) principle, where elements are added and removed from the same end.

LIFO Principle

- **Last In:** The most recently added element
- **First Out:** The first element to be removed
- **Access Point:** Only the top element is accessible
- **Analogy:** Stack of plates or books

Basic Stack Operations

- `Push`: Add element to top of stack
- `Pop`: Remove and return top element
- `Peek`/**Top:** View top element without removing
- **IsEmpty:** Check if stack has no elements

Stack Applications and Use Cases

Why Stacks are Important

Stacks are fundamental data structures with many practical applications in computing and everyday programming tasks.

Programming Applications

- Function call management
- Expression evaluation
- Syntax parsing
- Backtracking algorithms
- Memory management

User Interface Applications

- Undo operations
- Navigation history
- Dialog management
- State management

System Applications

- Browser back button
- OS scheduling

Algorithm Applications

- Depth-first search
- Tree traversal
- Graph algorithms

Stack Applications and Use Cases

Why Stacks are Important

Stacks are fundamental data structures with many practical applications in computing and everyday programming tasks.

Programming Applications

- Function call management
- Expression evaluation
- Syntax parsing
- Backtracking algorithms
- Memory management

User Interface Applications

- Undo operations
- Navigation history
- Dialog management
- State management

System Applications

- Browser back button
- OS scheduling

Algorithm Applications

- Depth-first search
- Tree traversal
- Graph algorithms

Stack Applications and Use Cases

Why Stacks are Important

Stacks are fundamental data structures with many practical applications in computing and everyday programming tasks.

Programming Applications

- Function call management
- Expression evaluation
- Syntax parsing
- Backtracking algorithms
- Memory management

User Interface Applications

- Undo operations
- Navigation history
- Dialog management
- State management

System Applications

- Browser back button
- OS scheduling

Algorithm Applications

- Depth-first search
- Tree traversal
- Graph algorithms

Study Case: Undo Operations

Problem

Implement an undo system for a text editor where users can reverse their recent changes in the order they were made.

```
1:  Structure EditAction
2:  actionType: String
3:  position: Integer
4:  oldText: String
5:  newText: String
6:  timestamp: DateTime
7:  end structure
8:
9:  Class UndoManager
10: undoStack: Stack of EditAction
11: redoStack: Stack of EditAction
12: maxUndoLevel: Integer
```





Study Case: Undo Operations

Problem

Implement an undo system for a text editor where users can reverse their recent changes in the order they were made.

```
1:  Structure EditAction
2:  actionType: String
3:  position: Integer
4:  oldText: String
5:  newText: String
6:  timestamp: DateTime
7:  end structure
8:
9:  Class UndoManager
10: undoStack: Stack of EditAction
11: redoStack: Stack of EditAction
12: maxUndoLevel: Integer
```





Study Case: Undo Operations

Problem

Implement an undo system for a text editor where users can reverse their recent changes in the order they were made.

```
1:  Structure EditAction
2:  actionType: String
3:  position: Integer
4:  oldText: String
5:  newText: String
6:  timestamp: DateTime
7:  end structure
8:
9:  Class UndoManager
10: undoStack: Stack of EditAction
11: redoStack: Stack of EditAction
12: maxUndoLevel: Integer
```

Undo System Operations

Study Case: Undo Operations

Problem

Implement an undo system for a text editor where users can reverse their recent changes in the order they were made.

```
1:  Structure EditAction
2:  actionType: String
3:  position: Integer
4:  oldText: String
5:  newText: String
6:  timestamp: DateTime
7:  end structure
8:
9:  Class UndoManager
10: undoStack: Stack of EditAction
11: redoStack: Stack of EditAction
12: maxUndoLevel: Integer
```

Undo System Operations

Queue: FIFO Principle and Operations

Definition

A queue is a linear data structure that follows the First-In-First-Out (FIFO) principle, where elements are added at one end and removed from the other end.

FIFO Principle

- **First In:** The earliest added element
- **First Out:** The first element to be removed
- **Two Ends:** Rear for insertion, front for removal
- **Analogy:** Line at a bank or cafeteria

Basic Queue Operations

- `Enqueue`: Add element to rear of queue
- `Dequeue`: Remove and return front element
- **Front:** View front element without removing

Queue: FIFO Principle and Operations

Definition

A queue is a linear data structure that follows the First-In-First-Out (FIFO) principle, where elements are added at one end and removed from the other end.

FIFO Principle

- **First In:** The earliest added element
- **First Out:** The first element to be removed
- **Two Ends:** Rear for insertion, front for removal
- **Analogy:** Line at a bank or cafeteria

Basic Queue Operations

- Enqueue: Add element to rear of queue
- Dequeue: Remove and return front element
- **Front:** View front element without removing

Queue: FIFO Principle and Operations

Definition

A queue is a linear data structure that follows the First-In-First-Out (FIFO) principle, where elements are added at one end and removed from the other end.

FIFO Principle

- **First In:** The earliest added element
- **First Out:** The first element to be removed
- **Two Ends:** Rear for insertion, front for removal
- **Analogy:** Line at a bank or cafeteria

Basic Queue Operations

- `Enqueue`: Add element to rear of queue
- `Dequeue`: Remove and return front element
- **Front:** View front element without removing

Queue: FIFO Principle and Operations

Definition

A queue is a linear data structure that follows the First-In-First-Out (FIFO) principle, where elements are added at one end and removed from the other end.

FIFO Principle

- **First In:** The earliest added element
- **First Out:** The first element to be removed
- **Two Ends:** Rear for insertion, front for removal
- **Analogy:** Line at a bank or cafeteria

Basic Queue Operations

- `Enqueue`: Add element to rear of queue
- `Dequeue`: Remove and return front element
- **Front:** View front element without removing

Queue Variations (Circular, Priority)

Types of Queues

Different queue implementations serve specific needs and optimize for different use cases.

Circular Queue

Structure:

- Fixed-size array
- Front and rear pointers
- Wrap-around mechanism

Benefits:

- Efficient memory usage
- No memory waste
- Fast operations
- Fixed memory footprint

Priority Queue

Structure:

- Elements have priorities
- Higher priority served first
- Can use heaps or sorted lists

Benefits:

- Serves important items first
- Flexible ordering
- Good for scheduling
- Real-world modeling

Queue Variations (Circular, Priority)

Types of Queues

Different queue implementations serve specific needs and optimize for different use cases.

Circular Queue

Structure:

- Fixed-size array
- Front and rear pointers
- Wrap-around mechanism

Benefits:

- Efficient memory usage
- No memory waste
- Fast operations
- Fixed memory footprint

Priority Queue

Structure:

- Elements have priorities
- Higher priority served first
- Can use heaps or sorted lists

Benefits:

- Serves important items first
- Flexible ordering
- Good for scheduling
- Real-world modeling

Queue Variations (Circular, Priority)

Types of Queues

Different queue implementations serve specific needs and optimize for different use cases.

Circular Queue

Structure:

- Fixed-size array
- Front and rear pointers
- Wrap-around mechanism

Benefits:

- Efficient memory usage
- No memory waste
- Fast operations
- Fixed memory footprint

Priority Queue

Structure:

- Elements have priorities
- Higher priority served first
- Can use heaps or sorted lists

Benefits:

- Serves important items first
- Flexible ordering
- Good for scheduling
- Real-world modeling

Queue Variations (Circular, Priority)

Types of Queues

Different queue implementations serve specific needs and optimize for different use cases.

Circular Queue

Structure:

- Fixed-size array
- Front and rear pointers
- Wrap-around mechanism

Benefits:

- Efficient memory usage
- No memory waste
- Fast operations
- Fixed memory footprint

Priority Queue

Structure:

- Elements have priorities
- Higher priority served first
- Can use heaps or sorted lists

Benefits:

- Serves important items first
- Flexible ordering
- Good for scheduling
- Real-world modeling

Study Case: Bank Office Queues

Problem

Implement a queue management system for a bank where customers are served in order of arrival, with special handling for different service types.

```
1:  Structure Customer
2:  customerId: Integer
3:  name: String
4:  serviceType: String
5:  arrivalTime: DateTime
6:  priority: Integer
7:  end structure
8:
9:  Class BankQueueSystem
10: regularQueue: Queue of Customer
11: priorityQueue: PriorityQueue of Customer
12: currentNumber: Integer
```





Study Case: Bank Office Queues

Problem

Implement a queue management system for a bank where customers are served in order of arrival, with special handling for different service types.

```
1: Structure Customer
2: customerId: Integer
3: name: String
4: serviceType: String
5: arrivalTime: DateTime
6: priority: Integer
7: end structure
8:
9: Class BankQueueSystem
10: regularQueue: Queue of Customer
11: priorityQueue: PriorityQueue of Customer
12: currentNumber: Integer
```





Study Case: Bank Office Queues

Problem

Implement a queue management system for a bank where customers are served in order of arrival, with special handling for different service types.

```
1: Structure Customer
2: customerId: Integer
3: name: String
4: serviceType: String
5: arrivalTime: DateTime
6: priority: Integer
7: end structure
8:
9: Class BankQueueSystem
10: regularQueue: Queue of Customer
11: priorityQueue: PriorityQueue of Customer
12: currentNumber: Integer
```

Queue System Operations

Study Case: Bank Office Queues

Problem

Implement a queue management system for a bank where customers are served in order of arrival, with special handling for different service types.

```
1: Structure Customer
2: customerId: Integer
3: name: String
4: serviceType: String
5: arrivalTime: DateTime
6: priority: Integer
7: end structure
8:
9: Class BankQueueSystem
10: regularQueue: Queue of Customer
11: priorityQueue: PriorityQueue of Customer
12: currentNumber: Integer
```

Queue System Operations

Linked-list Based Implementations I

Why Use Linked Lists for Stacks and Queues

Linked list implementations provide dynamic sizing and efficient operations for both stacks and queues.

Stack with Linked List

```
1: Class LinkedStack
2: top: Pointer to Node
3: size: Integer
4:
5: method push(element)
6: newNode =
   createNode(element)
7: newNode.next = top
8: top = newNode
9: size = size + 1
```

Queue with Linked List

```
1: Class LinkedQueue
2: front: Pointer to Node
3: rear: Pointer to Node
4: size: Integer
5:
6: method enqueue(element)
7: newNode =
   createNode(element)
8: rear.next = newNode
9: rear = newNode
10: size = size + 1
```

Linked-list Based Implementations I

Why Use Linked Lists for Stacks and Queues

Linked list implementations provide dynamic sizing and efficient operations for both stacks and queues.

Stack with Linked List

```
1: Class LinkedStack
2: top: Pointer to Node
3: size: Integer
4:
5: method push(element)
6: newNode = createNode(element)
7: newNode.next = top
8: top = newNode
9: size = size + 1
```

Queue with Linked List

```
1: Class LinkedQueue
2: front: Pointer to Node
3: rear: Pointer to Node
4: size: Integer
5:
6: method enqueue(element)
7: newNode = createNode(element)
8: rear.next = newNode
9: rear = newNode
10: size = size + 1
```

Linked-list Based Implementations II

Advantages of Linked Implementation

- **Dynamic Size:** No predefined limits
- **Memory Efficiency:** Allocate as needed
- **No Overflow:** Limited only by available memory
- **Flexible:** Easy to modify and extend

Considerations

- Extra memory overhead for pointers
- Dynamic memory allocation costs
- No random access to elements
- Potential memory fragmentation





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Linked-list Based Implementations II

Advantages of Linked Implementation

- **Dynamic Size:** No predefined limits
- **Memory Efficiency:** Allocate as needed
- **No Overflow:** Limited only by available memory
- **Flexible:** Easy to modify and extend

Considerations

- Extra memory overhead for pointers
- Dynamic memory allocation costs
- No random access to elements
- Potential memory fragmentation





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Complexity Analysis of Stack and Queue Operations I

Performance Characteristics

Understanding the time and space complexity helps choose the right implementation for specific needs.

Operation	Array-based	Linked-based	Notes
Stack `Push`	$O(1)$	$O(1)$	Constant time
Stack `Pop`	$O(1)$	$O(1)$	Constant time
Stack `Peek`	$O(1)$	$O(1)$	Constant time
Queue `Enqueue`	$O(1)$	$O(1)$	Constant time
Queue `Dequeue`	$O(1)$	$O(1)$	Constant time
Queue `Front`	$O(1)$	$O(1)$	Constant time





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Complexity Analysis of Stack and Queue Operations II

Space Complexity

- **Array-based:** $O(n)$ fixed allocation
- **Linked-based:** $O(n)$ dynamic allocation
- **Overhead:** Linked lists have pointer overhead

Performance Considerations

- All basic operations are constant time
- Array implementations may be faster in practice
- Linked implementations use memory more flexibly
- Choice depends on specific requirements





Complexity Analysis of Stack and Queue Operations II

Space Complexity

- **Array-based:** $O(n)$ fixed allocation
- **Linked-based:** $O(n)$ dynamic allocation
- **Overhead:** Linked lists have pointer overhead

Performance Considerations

- All basic operations are constant time
- Array implementations may be faster in practice
- Linked implementations use memory more flexibly
- Choice depends on specific requirements





Summary I

Key Concepts Covered

- **Lists:** Fundamental linear data structures with various implementations
- **ADTs:** Abstract Data Types provide interface-implementation separation
- **Advanced Lists:** `Double-linked`, `Circular`, and `Multi-list` structures
- **Stacks:** `LIFO` structures perfect for undo operations and function calls
- **Queues:** `FIFO` structures ideal for scheduling and service systems





Summary II

Implementation Choices

- `Static` vs. `Dynamic`: Consider size requirements
- Array vs. Linked: Consider access patterns
- Simple vs. Advanced: Consider functionality needs
- Performance trade-offs: Time vs. space considerations

Practical Applications

- Student enrollment systems using lists
- Browser navigation with `double-linked` lists
- Process scheduling with `circular` lists
- Undo systems with stacks
- Service queues with `FIFO` structures





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Summary II

Implementation Choices

- `Static` vs. `Dynamic`: Consider size requirements
- Array vs. Linked: Consider access patterns
- Simple vs. Advanced: Consider functionality needs
- Performance trade-offs: Time vs. space considerations

Practical Applications

- Student enrollment systems using lists
- Browser navigation with `double-linked` lists
- Process scheduling with `circular` lists
- Undo systems with stacks
- Service queues with `FIFO` structures





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Outline

1 Lists

2 Advanced List Structures

3 Stacks and Queues





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Thanks!

Questions?



Repo: *https://github.com/EngAndres/ud-public/tree/main/courses/computer-sciences-i*





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS